

TRI TOPOLOGIQUE

2

UTILITE :	2
RAPPEL :	2
LEMME.....	2
EXEMPLE 1	3
EXEMPLE 2	4
INTRODUCTION.....	5
REMARQUES.....	6
PRINCIPE DE L'ALGORITHME	8
<i>Exemple</i>	8
<i>Exemple</i>	10
<i>Implémentation Générique : Version itérative</i>	15
DETECTION DE CIRCUIT	16
<i>Implémentation Générique : Version itérative</i>	16
<i>Implémentation</i>	17
DETECTION DE CIRCUIT	20
<i>Exemple</i>	21
<i>Remarque</i>	23
UTILITE DU TRI TOPOLOGIQUE.....	23

TRI TOPOLOGIQUE

Utilité :

- Accélère certains algorithmes de Plus court chemin.
- Résout les problèmes de dépendances.
- Détecte la présence d'un circuit

Rappel :

Un graphe sans cycle est dit acyclique.

Lemme

Si un graphe simple orienté $G=(S, A)$ est **sans cycle**, alors
Il existe un sommet s tel que $d^-(s)=0$ (resp. $d^+(s)=0$)

//Algorithme permettant de décider si un graphe G est sans cycle.

```

Circuit ← faux
Tantque non vide(G) et non circuit faire
    s ← choisir un sommet sans prédécesseur
    si s n'existe pas alors
        circuit ← vrai
    fsi
// Mise à jour de la liste des prédécesseurs
    G ← G \ ({s}, {Arcs issus de s} )
Fintq

```

Exemple 1

Soit G un graphe dont les sommets décrivent des tâches à accomplir, et s'il existe un arc entre deux sommets t_i et t_j .

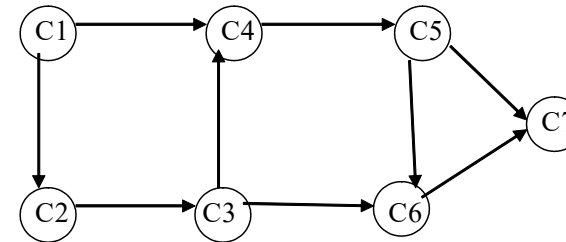
Alors, la tâche t_i doit être exécutée avant la tâche t_j (L'exécution d'un `makeFile` par exemple).

Plus généralement, si l'on considère que les sommets d'un graphe représentent des tâches et que chaque arc indique une dépendance,

Alors, le tri topologique indique un ordre des tâches qui respecte les dépendances.

Exemple 2

Considérons le graphe représentant un cursus universitaire suivant :



$C_1, \dots, C_7$ sont des cours à suivre :

Un arc du sommet C_i vers le sommet C_j



Veut dire « le cours C_i est préalable pour poursuivre le cours C_j »

Déterminer les cours qu'un étudiant doit poursuivre afin de respecter les contraintes de précédences.

Cet ordre est appelé un tri Topologique.

Introduction

L'opération qui consiste à trouver un ordre sur les sommets s'appelle un tri topologique :

Il s'agit de construire une liste de tous les sommets d'un graphe dans laquelle aucun sommet n'apparaît avant un de ses prédécesseurs.

Autrement dit, le tri topologique peut être vu comme un alignement des sommets de G le long d'une ligne horizontale de manière que tous les arcs soient orientés de gauche à droite.

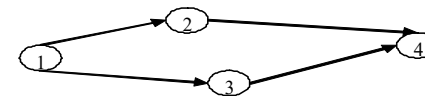
Remarques

❖ Un graphe orienté sans circuit sert souvent de représentation pour la relation d'ordre partiel $\leq$ sur les sommets, fondée sur la relation d'arc, qui vérifie la propriété suivante :

Il existe un arc de s vers t $\Rightarrow s \leq t$.

❖ Le tri topologique doit son nom au fait qu'il trie les sommets du graphe en respectant cette relation d'ordre.

Si l'on obtient plusieurs listes de solutions possibles pour un même graphe, c'est ce qu'il existe des sommets non comparables pour la relation $\leq$.

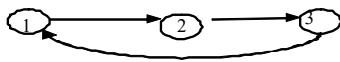


Par exemple, les sommets S_2 et S_3 dans le graphe ci-dessous sont non comparables :

$L_1 = \{1, 2, 4\}$ $L_2 = \{1, 3, 4\}$

➔ Le tri topologique a souvent plusieurs solutions, car on peut avoir à choisir entre plusieurs sommets dont tous les prédécesseurs sont déjà placés.

❖ Si le graphe n'est pas acyclique (comporte un cycle), aucun ordre linéaire n'est possible.



Principe de l'algorithme

Le tri topologique est une opération, qui étant donné un graphe orienté, il rend une liste de sommets.

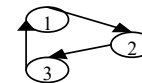
1) On sélectionne un sommet sans prédécesseur (le demi-degré intérieur du sommet est nul). On le retire du graphe, ainsi que tous les arcs qui en sont issus.

Lorsque plusieurs choix sont possibles ; on utilise l'ordre lexicographique par exemple pour sélectionner le sommet à traiter

2) Si le graphe est vide, le tri est terminé.

3) Si l'étape 1 n'a permis pas de sélectionner aucun sommet, c'est que le graphe comporte un circuit : le tri n'a pas de solution.

Exemple



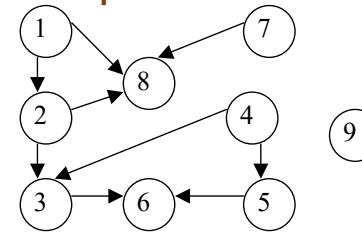
1 < 2 < 3 < 1

4) Recommencer à partir de l'étape 1.

Remarque

En utilisant le tri topologique, il est donc possible de construire un graphe où tous les arcs sont orientés dans le même sens.

Exemple



$$F = \{\}$$

$d^+(s) = \text{Card}(\Gamma^{-1}(s)) = \text{nombre de prédécesseur de } s.$

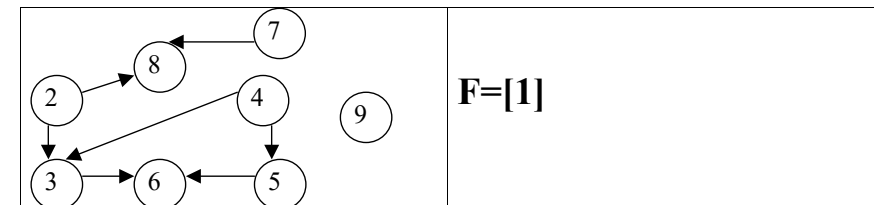
Sommets s	1	2	3	4	5	6	7	8	9
$\Gamma^{-1}(s)$	$\{\}$	$\{1\}$	$\{2,4\}$	$\{\}$	$\{4\}$	$\{3,5\}$	$\{\}$	$\{1, 2, 7\}$	$\{\}$
$d^+(s)$	0	1	2	0	1	2	0	3	0

Remarque : il existe des sommets sans prédécesseur →

G ne contient pas forcément un circuit. $L = [1, 2, 3, 4, 5, 6, 7, 8, 9]$

Sommets à traiter sont les sommets sans prédécesseur : 1 4 7 9

- **Traitement du Sommet 1** : on retire le sommet 1 ainsi que tous les arcs issus de 1

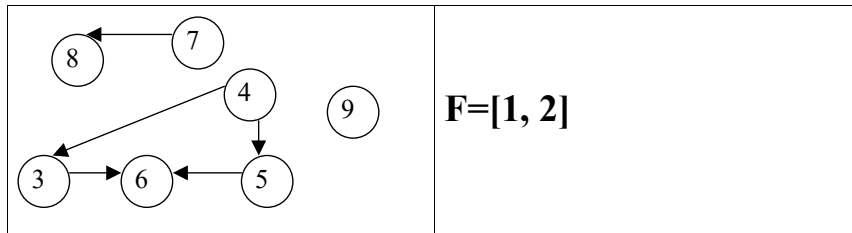


Sommets s	2	3	4	5	6	7	8	9
$\Gamma^{-1}(s)$	$\{\}$	$\{2,4\}$	$\{\}$	$\{4\}$	$\{3,5\}$	$\{\}$	$\{2, 7\}$	$\{\}$
$d^+(s)$	0	2	0	1	2	0	2	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 2 4 7 9

- **Traitement du Sommet 2** : on retire le sommet 2 ainsi que tous les arcs issus de 2

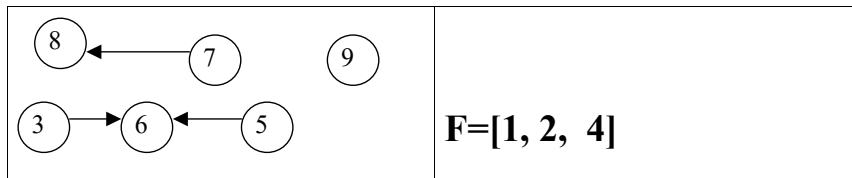


Sommets s	3	4	5	6	7	8	9
$\Gamma^{-1}(s)$	{4}	{}	{4}	{3,5}	{}	{7}	{}
$\text{Card}(\Gamma^{-1}(s))$	1	0	1	2	0	1	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 4 7 9

- **Traitement du Sommet 4** : on retire le sommet 4 ainsi que tous les arcs issus de 4

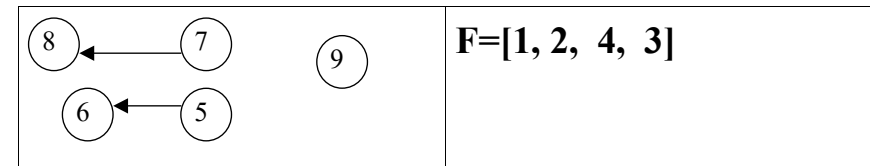


Sommets s	3	5	6	7	8	9
$\Gamma^{-1}(s)$	{}	{}	{3,5}	{}	{7}	{}
$\text{Card}(\Gamma^{-1}(s))$	0	0	2	0	1	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 3 5 7 9

- **Traitement du Sommet 3** : on retire le sommet 3 ainsi que tous les arcs issus de 3

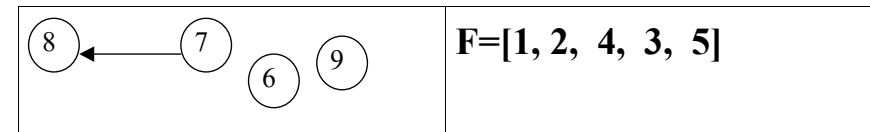


Sommets s	5	6	7	8	9
$\Gamma^{-1}(s)$	{}	{5}	{}	{7}	{}
$\text{Card}(\Gamma^{-1}(s))$	0	1	0	1	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 5 7 9

- **Traitement du Sommet 5** : on retire le sommet 5 ainsi que tous les arcs issus de 5

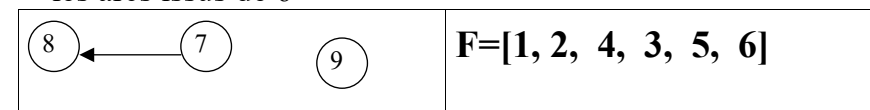


Sommets s	6	7	8	9
$\Gamma^{-1}(s)$	{}	{}	{7}	{}
$\text{Card}(\Gamma^{-1}(s))$	0	0	1	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 6 7 8 9

- **Traitement du sommet 6** : on retire le sommet 6 ainsi que tous les arcs issus de 6



Sommets s	7	8	9
$\Gamma^{-1}(s)$	$\{\}$	$\{7\}$	$\{\}$
$\text{Card}(\Gamma^{-1}(s))$	0	1	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 7, 8 9

- **Traitement du Sommet 7** : on retire le sommet 7 ainsi que tous les arcs issus de 7

8 9	$F=[1, 2, 4, 3, 5, 6, 7]$
---	---------------------------

Sommets s	8	9
$\Gamma^{-1}(s)$	$\{\}$	$\{\}$
$\text{Card}(\Gamma^{-1}(s))$	0	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 8 9

- **Traitement du Sommet 8** : on retire le sommet 8 ainsi que tous les arcs issus de 8

9	$F=[1, 2, 4, 3, 5, 6, 7, 8]$
---	------------------------------

Sommets s	9
$\Gamma^{-1}(s)$	$\{\}$
$\text{Card}(\Gamma^{-1}(s))$	0

On remarque qu'il existe des sommets sans prédécesseur

Sommets à traiter sont : 9

- **Traitement du Sommet 9** : on retire le sommet 9 ainsi que tous les arcs issus de 9

Graphe sans sommets.	$F=[1, 2, 4, 3, 5, 6, 7, 8, 9]$
-----------------------------	---------------------------------

- **FIN de l'algorithme**

Implémentation Générique : Version itérative

```

Fonction TriTopologique(G) → Liste ordonnée de sommets de G
P.F  G(S, A) : Graphe Acyclique
Début
/* Initialisation*/
F ← initFile()
estCircuit ← faux
Tantque non estVideGraphe(G) et non estCircuit faire
    Si tous les sommets ont un prédécesseur alors
        estCircuit ← vrai /*G contient un circuit*/
    Sinon
        /*Sélection d'un sommet sans prédécesseur */
        s ← Sommet Sans Prédécesseur
        /*Suppression du sommet s ainsi que les arcs issus de s */
        G ← G \ ({s}, (Arcs issus s))
        f ← enfiler(s, f)
    Finsi
Fintq
Si estCircuit alors
    Tantque non estVideF(f) faire
        f ← defiler(f)
    FinTq
FinSi
Renvoyer F
Fin

```

Détection de circuit

Implémentation Générique : Version itérative

```

Fonction TriTopologique(G, estCircuit) → Liste ordonnée de sommets de G
P.F  G(S, A) : Graphe (E)
        estCircuit : boolean (S)
Début
/* Initialisation*/
F ← initFile()
estCircuit ← faux
Tantque non estVideGraphe(G) et non estCircuit faire
    Si tous les sommets ont un prédécesseur alors
        estCircuit ← vrai /*G contient un circuit*/
    Sinon
        /*Sélection d'un sommet sans prédécesseur */
        s ← Sommet Sans Prédécesseur
        /*Suppression du sommet s ainsi que les arcs issus de s */
        G ← G \ ({s}, (Arcs issus s))
        f ← enfiler(s, f)
    Finsi
Fintq
Si estCircuit alors
    Tantque non estVideF(f) faire
        f ← defiler(f)
    FinTq
FinSi
Renvoyer F
Fin

```


Implémentation

Construire une suite de graphes obtenue par suppressions successives d'un sommet s tel que $\Gamma^{-1}(s)=\{\}$ (ou $d^{-1}(s)=0$).

En fait, on n'a pas besoin de construire complètement ces graphes :

On a seulement besoin de connaître à chaque étape de l'algorithme, les sommets qui restent et leurs demi-degrés intérieurs (qui a évolué en fonction des suppressions de sommets).

On associe donc à ces graphes successifs un tableau noté **d** indicé par les numéros de sommets, qui contient pour chaque sommet du graphe d'origine :

$d[s]=-1$ si le sommet a déjà été supprimé.

$d[s]$ = demi-degré intérieur mis à jour au fur et à mesure des suppressions.

Fonction triTopologique(G) → Liste ordonnée de sommets de G

$G(S, A)$: Graphe Acyclique

Début

// Phase initialisation

$n \leftarrow \text{nbSom}(G)$

$M \leftarrow \text{InitEnsVide}()$ // Contient les sommets sans prédécesseurs

$F \leftarrow \text{initFile}()$

Pour s variant de 1 à n faire

$de \leftarrow \text{degreEntrant}(s, G)$

si $de=0$ alors

$M \leftarrow \text{inser}(s, M)$

Fsi

$d[s] \leftarrow de$

fpour

// Traitement = Construction de la File F

Tant que non estVideEns(M) faire

$t \leftarrow \text{choisirUnSommet}(M)$

$M \leftarrow \text{supprimer}(t, M)$

$d[t] \leftarrow -1$

$f \leftarrow \text{enfiler}(t, f)$ //insert t ds une File (F) ou un tableau (F)

// Mise à jour du tableau d et de l'Ens M

Pour $u \in \Gamma(t)$ faire

$d[u] \leftarrow d[u] - 1$

si $d[u]=0$ alors

$M \leftarrow \text{inser}(u, M)$

Fsi

FinPour

FinTq

Renvoyer f

FIN

Détection de circuit

Fonction triTopologique(G, estCircuit) $\rightarrow$ Liste ordonnée de sommets de G

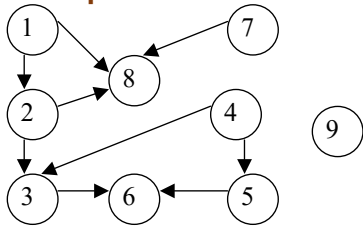
P.F $G : \text{Graphe (E)}$

$\text{estCircuit} : \text{Boolean (S)}$

Début

Voir TD

FIN

Exemple
 $d^+(s) = \text{Card}(\Gamma^{-1}(s)) = \text{nombre de prédécesseur de } s.$

Sommets s	1	2	3	4	5	6	7	8	9
$\Gamma^{-1}(s)$	$\{\}$	$\{1\}$	$\{2,4\}$	$\{\}$	$\{4\}$	$\{3,5\}$	$\{\}$	$\{1,2,7\}$	$\{\}$
$d^+(s)$	0	1	2	0	1	2	0	3	0
$\Gamma(s)$	$\{2,8\}$	$\{3,8\}$	$\{6\}$	$\{3,5\}$	$\{6\}$	$\{\}$	$\{8\}$	$\{\}$	$\{\}$

Phase Initialisation

Sommets s	1	2	3	4	5	6	7	8	9
d	0	1	2	0	1	2	0	3	0

 $M \leftarrow \{1, 4, 7, 9\} \quad F \leftarrow \{\}$
Itérations**Iter 1**
 $t \leftarrow 1 \quad F \leftarrow \{1\}$

Sommets s	1	2	3	4	5	6	7	8	9
d	-1	0	2	0	1	2	0	2	0

 $M \leftarrow \{4, 7, 9\}$
 $\text{Succ}(1) = \{2, 8\}$
 $M \leftarrow \{2, 4, 7, 9\}$
Iter 2
 $t \leftarrow 2 \quad F \leftarrow \{1, 2\}$

Sommets s	1	2	3	4	5	6	7	8	9
d	-1	-1	1	0	1	2	0	1	0

 $M \leftarrow \{4, 7, 9\}$
 $\text{Succ}(2) = \{3, 8\}$
Iter 3
 $t \leftarrow 4 \quad F \leftarrow \{1, 2, 4\}$

Sommets s	1	2	3	4	5	6	7	8	9
d	-1	-1	0	-1	0	2	0	1	0

 $M \leftarrow \{7, 9\}$
 $\text{Succ}(4) = \{3, 5\}$
ETC.....

Remarque

Selon la façon dont on représente le graphe et l'ensemble M, on obtient des algorithmes différents.

- Graphe représenté par matrice d'adjacence
- Graphe représenté par liste d'adjacence
- ...

Utilité du tri topologique

- Accélère certains algorithmes de recherches du plus court chemin.
- Résout les problèmes de dépendances.
- Détection la présence d'un circuit